

P.Murali Krishna  
(192525178)

## SIMATS ENGINEERING ASSESSMENT TOOL 1

**COURSE NAME:** Database Management Systems  
**COURSE CODE:** CSA05

---

### COURSE OUTCOME COVERED

**CO2:** *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)*

Activity Tool : Analytical Problem Solving (High)  
Assessment Tool Weightage: 35%

---

Dr

| QUESTIONS |                                                                                                                                                                             |               | Total Marks: 50 |
|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|-----------------|
| Q.No      | Question                                                                                                                                                                    | Bloom's Level | Marks           |
| 1         | Apply relational algebra operations ( $\sigma$ , $\pi$ , $\cup$ , $-$ , $\times$ , $\bowtie$ ) on the given Employee and Department relations to retrieve required records. | BL3 – Apply   | 5               |
| 2         | Write SQL DDL statements to create STUDENT(SID, Name, DOB, DeptID) and DEPARTMENT(DeptID, DeptName) tables with all appropriate constraints.                                | BL3 – Apply   | 5               |
| 3         | Formulate SQL queries using GROUP BY, HAVING, and aggregate functions (COUNT, SUM, AVG, MAX, MIN) on a given Sales dataset.                                                 | BL3 – Apply   | 5               |

|    |                                                                                                                                   |              |   |
|----|-----------------------------------------------------------------------------------------------------------------------------------|--------------|---|
| 4  | Write a correlated sub-query to find employees earning more than the average salary of their own department.                      | BL3 – Apply  | 5 |
| 5  | Apply all types of JOIN operations (INNER, LEFT OUTER, RIGHT OUTER, FULL OUTER, CROSS) on Employee and Project tables.            | BL3 – Apply  | 5 |
| 6  | Create a view 'DeptSalaryView' that displays department name, total employees, and average salary. Write a query on this view.    | BL6 – Create | 5 |
| 7  | Construct a relational algebra expression for the following: Find names of students who are enrolled in all courses offered.      | BL6 – Create | 5 |
| 8  | Write SQL DML statements (INSERT, UPDATE, DELETE) with integrity constraint enforcement for a given scenario.                     | BL3 – Apply  | 5 |
| 9  | Apply tuple relational calculus to express a query: Find all employees who work in the 'IT' department and earn more than 50000.  | BL3 – Apply  | 5 |
| 10 | Design a relational schema for an Airline Reservation System and write 5 SQL queries demonstrating joins, sub-queries, and views. | BL6 – Create | 5 |

---

**Code of Conduct:**

*I P.Murali Krishna(192525178) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

*I understand that any violation of academic integrity rules will result in disciplinary action.*

**Signature of the student:**



## RUBRICS FOR CALCULATION:

| Numerical Problems        |           |                                                                                   |                                           |                                             |                                               |
|---------------------------|-----------|-----------------------------------------------------------------------------------|-------------------------------------------|---------------------------------------------|-----------------------------------------------|
| Criteria                  | Weightage | Level 4 (Exemplary)                                                               | Level 3 (Proficient)                      | Level 2 (Developing)                        | Level 1 (Beginning)                           |
| Problem Understanding     | 20%       | Clearly interprets problem, identifies all parameters and requirements accurately | Understands problem with minor gaps       | Partial understanding ; misses key elements | Misinterprets or unable to understand problem |
| Method Selection          | 20%       | Selects most appropriate method/formula with strong justification                 | Selects correct method with minor errors  | Inappropriate or partially correct method   | Unable to select method                       |
| Solution Process          | 25%       | Logical, systematic steps with correct approach throughout                        | Mostly correct steps with minor mistakes  | Disorganized steps; multiple errors         | No clear process                              |
| Accuracy of Calculation   | 20%       | All calculations accurate; correct final answer                                   | Minor calculation errors                  | Frequent errors; incorrect final answer     | No valid calculations                         |
| Interpretation of Results | 15%       | Clearly interprets results with units and engineering relevance                   | Basic interpretation with minor omissions | Limited or unclear interpretation           | No interpretation                             |

## Sample Relations Used

### Employee

| EmpID | Name         | Salary | DeptID | ManagerID |
|-------|--------------|--------|--------|-----------|
| E101  | Rahul Sharma | 75000  | D1     | E105      |
| E102  | Priya Patel  | 62000  | D2     | E106      |
| E103  | Amit Kumar   | 55000  | D1     | E105      |
| E104  | Sneha Reddy  | 48000  | D3     | E107      |

|      |              |       |    |      |
|------|--------------|-------|----|------|
| E105 | Vikram Singh | 95000 | D1 | NULL |
| E106 | Ananya Iyer  | 88000 | D2 | NULL |
| E107 | Karthik Nair | 72000 | D3 | NULL |
| E108 | Meera Joshi  | 51000 | D2 | E106 |

## Department

| DeptID | DeptName  | Location  |
|--------|-----------|-----------|
| D1     | IT        | Hyderabad |
| D2     | HR        | Chennai   |
| D3     | Finance   | Bangalore |
| D4     | Marketing | Mumbai    |

## Project

| ProjID | ProjName           | DeptID | Budget |
|--------|--------------------|--------|--------|
| P1     | ERP System         | D1     | 500000 |
| P2     | Recruitment Portal | D2     | 150000 |
| P3     | Payroll Module     | D3     | 200000 |
| P4     | Mobile App         | D1     | 350000 |

## Works\_On

| EmpID | ProjID | Hours |
|-------|--------|-------|
| E101  | P1     | 40    |
| E101  | P4     | 20    |
| E103  | P1     | 35    |
| E105  | P1     | 10    |
| E105  | P4     | 15    |
| E102  | P2     | 30    |
| E104  | P3     | 25    |

## Q1. Apply Relational Algebra Operations ( $\sigma$ , $\pi$ , $\cup$ , $-$ , $\times$ , $\bowtie$ ) (5 Marks)

**a) Names of employees in IT department earning > 60000**

$\pi \text{ Name } (\sigma \text{ Salary} > 60000 \wedge \text{DeptName} = \text{'IT'} (\text{Employee} \bowtie \text{Department}))$

Result: Rahul Sharma, Vikram Singh

**b) Employees who are not managers (set difference)**

$\pi \text{ EmpID } (\text{Employee}) - \pi \text{ ManagerID } (\sigma \text{ ManagerID} \neq \text{NULL} (\text{Employee}))$

**c) Cartesian product filtered to matching DeptID**

$\sigma \text{ Employee.DeptID} = \text{Department.DeptID} (\text{Employee} \times \text{Department})$

**d) Natural join**

$\text{Employee} \bowtie \text{Department}$

**e) Union of employees in IT and Finance**

$\pi \text{ Name } (\sigma \text{ DeptID} = \text{'D1'} (\text{Employee})) \cup \pi \text{ Name } (\sigma \text{ DeptID} = \text{'D3'} (\text{Employee}))$

## Q2. SQL DDL – STUDENT and DEPARTMENT tables (5 Marks)

```
CREATE TABLE DEPARTMENT (  
    DeptID   VARCHAR(10) PRIMARY KEY,  
    DeptName VARCHAR(50) NOT NULL UNIQUE  
);  
CREATE TABLE STUDENT (  
    SID      VARCHAR(10) PRIMARY KEY,  
    Name     VARCHAR(50) NOT NULL,  
    DOB      DATE,  
    DeptID   VARCHAR(10),
```

```
CONSTRAINT fk_dept FOREIGN KEY (DeptID)
REFERENCES DEPARTMENT(DeptID)
ON DELETE SET NULL
ON UPDATE CASCADE,
CONSTRAINT chk_dob CHECK (DOB < CURRENT_DATE)
```

);

Constraints: PRIMARY KEY, FOREIGN KEY (with referential actions), NOT NULL, UNIQUE, CHECK.

### Q3. SQL with GROUP BY, HAVING & Aggregates (5 Marks)

Assume Sales(SaleID, Product, Category, Amount, SaleDate)

```
SELECT Category, SUM(Amount) AS TotalSales
FROM Sales
GROUP BY Category;
```

```
SELECT Category, SUM(Amount) AS TotalSales
FROM Sales
GROUP BY Category
HAVING SUM(Amount) > 10000;
```

```
SELECT Category,
       COUNT(*) AS NumSales,
       AVG(Amount) AS AvgSale,
       MAX(Amount) AS MaxSale,
       MIN(Amount) AS MinSale
FROM Sales
GROUP BY Category;
```

### Q4. Correlated Sub-query – Employees > Dept Average Salary (5 Marks)

```
SELECT E1.EmpID, E1.Name, E1.Salary, E1.DeptID
FROM Employee E1
WHERE E1.Salary > (
    SELECT AVG(E2.Salary)
    FROM Employee E2
    WHERE E2.DeptID = E1.DeptID
);
```

Correlated because inner query references outer E1.DeptID. Expected: Vikram Singh, Ananya Iyer, Karthik Nair.

### Q5. All Types of JOIN Operations (5 Marks)

```
-- INNER JOIN
SELECT E.Name, P.ProjName
FROM Employee E
INNER JOIN Works_On W ON E.EmpID = W.EmpID
```

```

INNER JOIN Project P ON W.ProjID = P.ProjID;

-- LEFT OUTER JOIN
SELECT E.Name, P.ProjName
FROM Employee E
LEFT JOIN Works_On W ON E.EmpID = W.EmpID
LEFT JOIN Project P ON W.ProjID = P.ProjID;

-- RIGHT OUTER JOIN
SELECT E.Name, P.ProjName
FROM Employee E
RIGHT JOIN Works_On W ON E.EmpID = W.EmpID
RIGHT JOIN Project P ON W.ProjID = P.ProjID;

-- FULL OUTER JOIN
SELECT E.Name, P.ProjName
FROM Employee E
FULL OUTER JOIN Works_On W ON E.EmpID = W.EmpID
FULL OUTER JOIN Project P ON W.ProjID = P.ProjID;

-- CROSS JOIN
SELECT E.Name, P.ProjName
FROM Employee E CROSS JOIN Project P;

```

## Q6. Create View DeptSalaryView & Query (5 Marks)

```

CREATE VIEW DeptSalaryView AS
SELECT D.DeptName,
       COUNT(E.EmpID) AS TotalEmployees,
       AVG(E.Salary) AS AvgSalary
FROM Department D
LEFT JOIN Employee E ON D.DeptID = E.DeptID
GROUP BY D.DeptName;

SELECT * FROM DeptSalaryView
WHERE AvgSalary > 60000
ORDER BY AvgSalary DESC;

```

## Q7. Relational Algebra – Students enrolled in ALL courses (5 Marks)

Assume: Student(SID, Name), Course(CID, CName), Enrolled(SID, CID)

$\pi \text{ SID, Name (Student)} \div \pi \text{ CID (Course)}$

-- Expanded form:  
 $\pi \text{ Name (Student)} -$

```

π Name (
  (π SID, Name (Student) × π CID (Course))
  − π SID, Name, CID (Student ⋈ Enrolled)
)

```

Division finds students for whom there is no course they are not enrolled in.

## Q8. SQL DML with Integrity Constraint Enforcement (5 Marks)

```

INSERT INTO Employee (EmpID, Name, Salary, DeptID, ManagerID)
VALUES ('E109', 'Arjun Mehta', 58000, 'D1', 'E105');

-- This fails FK: INSERT INTO Employee VALUES ('E110','Test',40000,'D9',NULL);

UPDATE Employee SET Salary = Salary * 1.10
WHERE DeptID = 'D2' AND Salary < 60000;

DELETE FROM Employee WHERE DeptID = 'D4';
DELETE FROM Department WHERE DeptID = 'D4';

```

## Q9. Tuple Relational Calculus – IT employees earning > 50000 (5 Marks)

```

{ e.Name, e.Salary | Employee(e) ∧ e.Salary > 50000
  ∧ ∃ d (Department(d) ∧ d.DeptID = e.DeptID ∧ d.DeptName = 'IT') }

```

## Q10. Airline Reservation System Schema + 5 SQL Queries (5 Marks)

### Schema

```

Airport(AirportCode PK, Name, City, Country)
Airline(AirlineID PK, Name, Country)
Flight(FlightNo PK, AirlineID FK, SourceAirport FK, DestAirport FK,
       DepartureTime, ArrivalTime, Capacity)
Passenger(PassengerID PK, Name, Email, Phone, PassportNo UNIQUE)
Booking(BookingID PK, PassengerID FK, FlightNo FK, BookingDate,
        SeatNo, Status, Amount)
Payment(PaymentID PK, BookingID FK, PaymentDate, Method, Amount)

```

### 5 SQL Queries

```

-- 1. JOIN: passengers with flight & airline
SELECT P.Name, F.FlightNo, A.Name AS Airline, F.DepartureTime
FROM Passenger P
JOIN Booking B ON P.PassengerID = B.PassengerID
JOIN Flight F ON B.FlightNo = F.FlightNo
JOIN Airline A ON F.AirlineID = A.AirlineID;

```



-- 2. Sub-query: flights with no bookings

```
SELECT FlightNo, SourceAirport, DestAirport FROM Flight
WHERE FlightNo NOT IN (SELECT DISTINCT FlightNo FROM Booking);
```

-- 3. Nested sub-query: passengers on most expensive booking

```
SELECT Name FROM Passenger WHERE PassengerID IN (
  SELECT PassengerID FROM Booking
  WHERE Amount = (SELECT MAX(Amount) FROM Booking));
```

-- 4. View: daily booking summary

```
CREATE VIEW DailyBookingSummary AS
SELECT BookingDate, COUNT(*) AS TotalBookings, SUM(Amount) AS Revenue
FROM Booking GROUP BY BookingDate;
SELECT * FROM DailyBookingSummary WHERE Revenue > 100000;
```

-- 5. Multi-join + aggregate

```
SELECT A.Name, AVG(B.Amount) AS AvgBookingValue
FROM Airline A
JOIN Flight F ON A.AirlineID = F.AirlineID
JOIN Booking B ON F.FlightNo = B.FlightNo
GROUP BY A.Name HAVING AVG(B.Amount) > 5000;
```